

Subsection 3.2, we also have $e(\mathcal{H}) \geq n$. Consequently,

$$\text{sat}_3(n, \text{Berge-}K_4) = n \text{ holds for } n \geq 96.$$

For smaller n , a computer search yields $\text{sat}_3(6, \text{Berge-}K_4) = 5$ and $\text{sat}_3(n, \text{Berge-}K_4) = n$ for $n = 5, 7, 8$ (see Appendix A). Furthermore, by applying Lemma 2.1, we can generate numerous non-isomorphic families of extremal hypergraphs by adding $\mathcal{T}$ on various pairs of vertices in the hypergraphs listed in Table 1, which completes the proof of Theorem 1.1. $\blacksquare$

4 Algorithm for finding uniform Berge- K_ℓ saturated hypergraphs

In this section, we introduce an algorithm for finding k -uniform Berge- K_ℓ saturated hypergraphs with given number of vertices and hyperedges, where k and ℓ are non-negative integers. For expository convenience, we take $k = 3$ and $\ell = 4$. Let's recall that the incidence graph of a hypergraph is a bipartite graph with one part representing the hypergraph's vertices and the other representing its hyperedges, where an edge connects a vertex to a hyperedge if the vertex belongs to that hyperedge.

The algorithm is straightforward. We first use `generate_hypergraphs()` to generate all candidate hypergraphs via backtracking and get their incidence graphs, then use `select_berge_k4_saturated_parallel()` to select the ones corresponding to Berge- K_4 -saturated hypergraphs in parallel, finally use `non_isomorphic()` and `graph_to_hyper()` to obtain non-isomorphic ones in parallel and convert them back to hypergraphs.

Algorithm 1 Searching 3-Uniform Berge K_4 Saturated Hypergraphs

Input: $n_vertices$, n_edges , $uniform$, n_min_degree

Output: A list of non-isomorphic Berge K_4 saturated hypergraphs

```

    // Generate all candidate hypergraphs via backtracking, and convert them into
    incidence graphs.
1: candidates  $\leftarrow$  generate_hypergraphs( $n\_vertices$ ,  $n\_edges$ ,  $uniform$ ,  $n\_min\_degree$ );
    // Parallel selection: for each  $g \in$  candidates,
    is_berge_k4_saturated( $g$ ) checks is_berge_k4_free( $g$ ) first,
    then checks saturation.
2: berge_k4_saturated  $\leftarrow$ 
    select_berge_k4_saturated_parallel(candidates);
3: non_isomorphic  $\leftarrow$  non_isomorphic(berge_k4_saturated);
4: result  $\leftarrow$  graph_to_hyper(non_isomorphic);
5: return result;
```

Considering the incidence graphs allows us to reduce the problem of deciding whether a hypergraph is Berge- K_4 -saturated to the problem of finding maximum matchings in auxiliary graphs derived from its incidence graph. Moreover, it enables us to select non-isomorphic hypergraphs by using VF2++ algorithm [15], which is an algorithm for graph isomorphism detection.

We now turn to `select_berge_k4_saturated_parallel()`, which is a wrapper to execute `is_berge_k4_saturated()` in parallel. Generally speaking, `is_berge_k4_saturated()` first uses `is_berge_k4_f`

`ree()` to check whether a hypergraph is Berge- K_4 -free, if so, it further tests whether the hypergraph is Berge- K_4 -saturated. Let's introduce `is_berge_k4_free()` first. For ease of illustration, we set the input to be a hypergraph (although it is actually an incidence graph).

Let $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ be the input hypergraph, we consider the 4-element subsets of $\mathcal{V}$. If there is a 4-element subset that forms a Berge- K_4 , then $\mathcal{H}$ is not Berge- K_4 -free; otherwise, it is. For each 4-element subset T , we construct an auxiliary bipartite graph with $L := \{p : p \subseteq T, |p| = 2\}$ as its left part and $R := \mathcal{E}$ as its right part. For $p \in L$ and $e \in R$, p is adjacent to e if and only if $p \subseteq e$. It is easy to see that if the matching number of the auxiliary bipartite is 6, then T forms a Berge- K_4 implies $\mathcal{H}$ is not Berge- K_4 -free; otherwise, T can not form a Berge- K_4 . This is how `is_berge_k4_free()` works.

Algorithm 2 `is_berge_k4_free( $\mathcal{H}$ )`

Input: A hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$

Output: `True` if $\mathcal{H}$ is Berge K_4 -free; `False` otherwise

```

1: for each 4-tuple  $T = (v_1, v_2, v_3, v_4) \subseteq \mathcal{V}$  with  $|T| = 4$  do
2:   Let  $P_T \leftarrow$  all  $\binom{4}{2} = 6$  unordered pairs from  $T$ ;
3:   Construct a bipartite graph  $\text{aux\_g} = (L \cup R, E)$  where:
      •  $L \leftarrow P_T$  (left part: the 6 pairs),
      •  $R \leftarrow \mathcal{E}$  (right part: all hyperedges of  $H$ ),
      •  $(p, e) \in E$  iff  $p \subseteq e$  (i.e., pair  $p$  is contained in hyperedge  $e$ ).

      // Compute a maximum matching  $M$  in  $\text{aux\_g}$ .
4:    $M \leftarrow \text{nx.max\_weight\_matching}(\text{aux\_g}, \text{maxcardinality}=\text{True})$ ;
5:   if  $|M| = 6$  then
6:     return False // Berge  $K_4$  found
7:   end if
8: end for
9: return True //  $H$  is Berge  $K_4$ -free

```

Next, we introduce `is_berge_k4_saturated()`. In this algorithm, we first check whether $\mathcal{H}$ is Berge- K_4 -free. For each Berge- K_4 -free hypergraph, we find out all its bad vertex pairs. If there are 3 bad vertex pairs form a non-hyperedge, then by the definition of bad pair, adding this non-hyperedge does not create a Berge- K_4 , thereby implying that the hypergraph is not Berge- K_4 -saturated; otherwise, it is.

Recall that a vertex pair (v_1, v_2) is defined as good if there exists two vertices v_3, v_4 and five hyperedges $e_{13}, e_{14}, e_{23}, e_{24}, e_{34}$ such that $\{v_i, v_j\} \subseteq e_{i,j}$ for all $(i, j) \neq (1, 2)$. Furthermore, v_3, v_4 are common neighbors of v_1 and v_2 with each having degree at least 3. Inspired by this, for a vertex pair (u, v) , we consider all 4-element subsets $\{u, v, x, y\}$ where x, y are two common neighbors of u and v with each having degree at least 3. We construct auxiliary bipartite graphs similar to those in Algorithm 2 and examine their matching numbers to determine whether (u, v) is good.

Algorithm 3 `is_berge_k4_saturated( $\mathcal{H}$ )`

Input: A hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$

Output: True if $\mathcal{H}$ is Berge K_4 -saturated; False otherwise

```

1: if is_berge_k4_free( $\mathcal{H}$ ) = False then
2:   return False
3: end if                                     // Must be Berge  $K_4$ -free to be saturated
4: badPairs  $\leftarrow \emptyset$ ;
5: for each unordered pair  $(u, v)$  with  $u \neq v$  do
6:   commonNeighbors  $\leftarrow$ 
        $\{w \in \mathcal{V} \setminus \{u, v\} \mid w \text{ adjacent to both } u \text{ and } v, \deg(w) \geq 3\}$ ;
7:   if |commonNeighbors| < 3 then
8:     add  $(u, v)$  to badPairs;
9:     continue;
10:  end if
11:  isGood  $\leftarrow$  False;
12:  for each unordered pair  $(x, y) \subseteq$  commonNeighbors do
13:    Let  $T \leftarrow (u, v, x, y)$ ;
14:    Let  $P_T \leftarrow$  all  $\binom{4}{2} - 1 = 5$  pairs from  $T$  excluding  $(u, v)$ ;
15:    Construct bipartite graph aux_g =  $(L \cup R, E)$  where:
        •  $L \leftarrow P_T$ ,
        •  $R \leftarrow \mathcal{E}$ ,
        •  $(p, e) \in E$  iff  $p \subseteq e$ .
16:    Compute a maximum matching  $M$  in aux_g;
17:    if  $|M| = 5$  then
18:      isGood  $\leftarrow$  True;
19:      break;                                     // Found one witness:  $(u, v)$  is good
20:    end if
21:  end for
22:  if not isGood then
23:    add  $(u, v)$  to badPairs;
24:  end if
25: end for
    // Check saturation condition: every non-edge must create a Berge  $K_4$ .
26: for each triple of distinct bad pairs  $\{(a, b), (b, c), (a, c)\}$  do
27:   if  $\{a, b, c\}$  induces a non-hyperedge in  $H$  then
28:     return False;
29:   end if
30: end for
31: return True;

```
